Alaws lang.

CLI-terminal video analysis chat app - product and implementation plan

Primary UI style	Minimal CLI-terminal interface with a Codex-like composer feel.
Core behavior	User pastes a video URL inside the prompt. The app analyzes the video and returns terminal-style output.
Frontend	Next.js hosted on Vercel.
Backend proxy	Cloudflare Worker hides API keys and handles provider fallback.
Storage v1	Browser localStorage for free local chat history and settings.
Primary AI provider	NVIDIA NIM.
Fallback provider	OpenRouter free Gemma models.

This document is written as a build reference for Codex. It combines the architecture, UI rules, model list, fallback logic, storage plan, custom instructions, and implementation checklist for the first version of Alaws lang.

1. Product goal

Alaws lang. is a minimal CLI-style web app for video analysis. The user pastes a video URL directly into the prompt, asks a question, and receives a structured video summary, key moments, warnings, and action items.

User opens site
-> User pastes video URL inside prompt
-> User asks what to analyze
-> Cloudflare Worker calls the selected AI model
-> AI returns terminal-style analysis
-> Chat is saved locally in browser

Example user prompt

https://cdn.example.com/video.mp4

Analyze this video.
Give me:
- short summary
- key moments with timestamps
- warnings or important events
- action items

2. Tech stack

Frontend	Next.js hosted on Vercel.
Backend	Cloudflare Worker used as API proxy and provider router.
Storage v1	localStorage. No database needed for the free first version.
Primary AI	NVIDIA NIM models.
Fallback AI	OpenRouter free Gemma models.
Future storage	Cloudflare D1, Supabase, Firebase, or Postgres if login/sync is added later.

Next.js UI on Vercel
|
v
Cloudflare Worker API
|
v
Provider router / fallback logic
|
+--> NVIDIA NIM
|
+--> OpenRouter
|
v
Return analysis to chat UI

Important: API keys must never be exposed in the browser. Store them in Cloudflare Worker environment variables only.

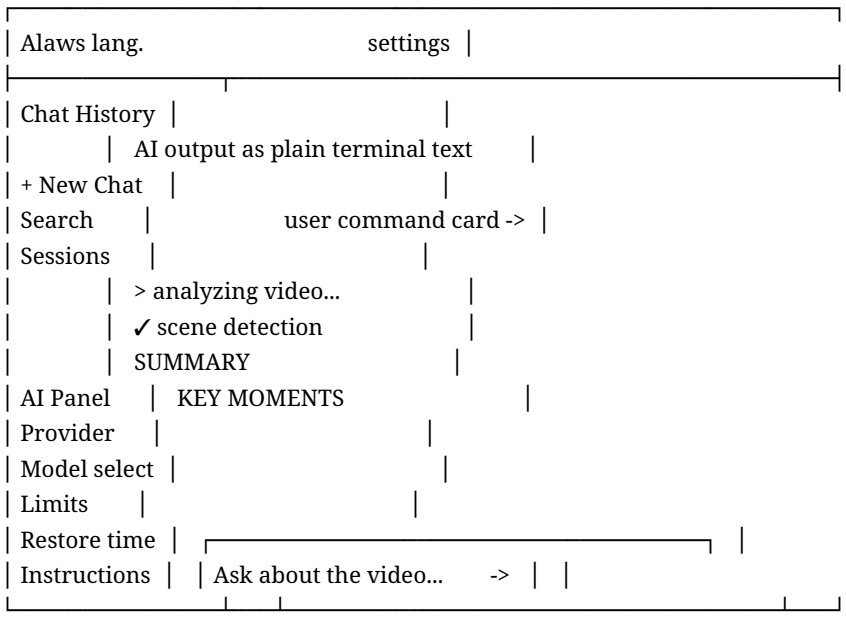
```
NVIDIA_API_KEY
OPENROUTER_API_KEY
```

3. UI direction

The UI should feel like a clean terminal and developer composer, not a heavy analytics dashboard. It should be readable, quiet, and simple.

Base colors	Black, white, and gray.
Green	Success, active state, completed steps.
Orange	Warnings, processing, low remaining limits.
Red	Errors, failed provider, exhausted limits.
Typography	Monospace for transcript, prompts, timestamps, and AI output. Sans-serif can be used for navigation labels.
Avoid	Heavy dashboards, too many colors, large analytics widgets, AI chat bubbles, extra labels in the composer.

4. Final layout



5. Sidebar design

The sidebar has two vertical sections. The first section is chat history. The second section is an AI panel that replaces the normal user card.

First row: chat history

```
+ New Chat
Search chats...
```

Q2 Product Demo Breakdown
Factory Floor Safety Review
Interview Highlights
Bug Repro Screen Capture
Security Camera Summary

Second row: AI panel

AI

Provider
NVIDIA NIM

Model
cosmos-reason2-8b

Limits
47 / 50 remaining

Restores
in 18m 24s

Status
active

[Instructions]

The AI panel should contain provider/model selection, current rate limit state, restore time, status, and the instructions button. Keep these controls out of the composer.

6. Main chat behavior

User messages

User messages are right-aligned command cards. They can contain the video URL and the question. Use a subtle border and dark card background. The user card is the only chat-card style message in the transcript.

[user]
https://cdn.example.com/factory-demo.mp4

Analyze this video and find:
- key moments
- warnings
- action items

AI messages

AI responses are not chat bubbles. Render them as plain terminal output aligned to the left/main content area.

> analyzing video...
✓ audio transcription
✓ scene detection
✓ key moment extraction

✓ summarization complete

SUMMARY
The video shows a factory floor during normal operation.
A worker enters a restricted area at 03:12 and a machine stops at 06:40.

KEY MOMENTS
[00:00] Video begins
[01:20] Workers enter frame
[03:12] Warning: restricted zone entry
[06:40] Machine downtime starts
[08:02] Machine returns to normal

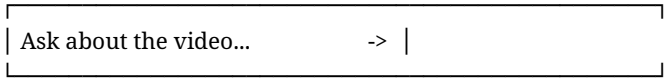
WARNINGS
orange: restricted zone entry at 03:12
red: machine jam detected at 06:40

ACTION ITEMS
- Review restricted zone signage
- Check machine maintenance logs
- Export clip around 03:12

7. Composer design

The bottom composer must stay extremely simple. The user includes the video URL directly in the prompt, so there is no separate video URL field.

Include	One multiline chatbox and one send icon.
Do not include	Separate video URL field, model selector, rate limits, restore time, extra labels, dashboard controls.
Placeholder	Ask about the video...



8. Custom instructions

The app should allow the user to customize the AI behavior. Put this behind an Instructions button in the sidebar AI panel. Use a modal or drawer.

System Instructions

[textarea]

Save
Reset to default

Default system instructions

You are a video analysis assistant.
Analyze the provided video URL carefully.

Return clear, structured output with:
- short summary
- key moments with timestamps
- warnings or important events
- action items
Keep the response concise and useful.

Instruction presets

General Summary
Security Review
Meeting/Demo Summary
Safety Inspection
Object/Event Detection

Prompt construction

System instructions:
[user customized instructions]

User prompt:
[user prompt including video URL]

9. localStorage plan

Use localStorage for the first version. It is free and enough for a prototype.

Store	Chat sessions, messages, selected provider/model, custom instructions, local settings, last prompt.
Do not store	API keys, large video files, sensitive user data.
Limitation	Data stays only on the same browser/device and can be cleared by the user.

localStorage keys:
- alaws.sessions
- alaws.activeSessionId
- alaws.settings
- alaws.instructions
- alaws.aiPanelState

Suggested session shape

```
type ChatSession = {
  id: string;
  title: string;
  createdAt: string;
  updatedAt: string;
  messages: Message[];
};

type Message = {
  id: string;
  role: 'user' | 'assistant' | 'system';
  content: string;
```

```
  createdAt: string;
  provider?: 'nvidia' | 'openrouter';
  model?: string;
  status?: 'queued' | 'running' | 'done' | 'error';
};
```

10. AI providers and models

Use NVIDIA NIM first for direct video analysis. Use OpenRouter as fallback, especially for free Gemma models. If a fallback model cannot process raw video input, add frame extraction later.

Provider	Model	Role in app
NVIDIA NIM	cosmos-reason2-8b	Primary video-analysis model.
NVIDIA NIM	Qwen3.5-397B-A17B	High-capability NVIDIA fallback.
NVIDIA NIM	Qwen3.5-122B-A10B	Strong NVIDIA fallback.
NVIDIA NIM	Gemma 4 31B IT	NVIDIA Gemma fallback.
NVIDIA NIM	gemma-3n-e4b-it	Smaller/cheaper NVIDIA fallback.
OpenRouter	google/gemma-4-31b-it:free	Free fallback.
OpenRouter	google/gemma-4-26b-a4b-it:free	Free fallback.

11. Recommended fallback order

1. NVIDIA NIM: cosmos-reason2-8b
2. NVIDIA NIM: Qwen3.5-122B-A10B
3. NVIDIA NIM: Qwen3.5-397B-A17B
4. NVIDIA NIM: Gemma 4 31B IT
5. NVIDIA NIM: gemma-3n-e4b-it
6. OpenRouter: google/gemma-4-31b-it:free
7. OpenRouter: google/gemma-4-26b-a4b-it:free

Provider router behavior

User sends prompt
-> Try selected model first
-> If rate-limited, try next fallback model
-> If provider fails, try second provider
-> If all providers fail, return a clean user-facing error

Clean error messages

All providers are currently busy. Try again in a few minutes.

This model does not support this video input. Try another model.

The video URL could not be accessed. Make sure it is public or signed.

12. Cloudflare Worker API contract

Frontend request

```
POST /api/analyze
Content-Type: application/json

{
  "sessionId": "session_123",
  "prompt": "https://cdn.example.com/video.mp4

Analyze this video...",
  "provider": "nvidia",
  "model": "cosmos-reason2-8b",
  "instructions": "You are a video analysis assistant...",
  "fallback": true
}
```

Worker response

```
{
  "ok": true,
  "provider": "nvidia",
  "model": "cosmos-reason2-8b",
  "content": "> analyzing video...

SUMMARY
...",
  "usage": {
    "requestsRemaining": 47,
    "restoreTime": "in 18m 24s"
  }
}
```

Worker responsibilities

- Read NVIDIA_API_KEY and OPENROUTER_API_KEY from Worker env vars.
- Build the provider request.
- Try selected model first.
- Apply fallback order when enabled.
- Normalize response into one simple output format.
- Return clean errors.
- Never leak raw provider API errors to the UI.

13. Suggested Next.js component tree

```
app/
  page.tsx
  layout.tsx
  globals.css

components/
  AppShell.tsx
  Sidebar.tsx
```

```
  ChatHistory.tsx
  AiPanel.tsx
  InstructionsModal.tsx
  Transcript.tsx
  UserCommandCard.tsx
  AssistantTerminalOutput.tsx
  Composer.tsx

lib/
  storage.ts
  models.ts
  api.ts
  format.ts
  prompts.ts

types/
  chat.ts
  models.ts
```

14. Visual implementation rules

1. Composer has only one text box and one send icon.
2. The video URL is part of the prompt text, not a separate field.
3. The model selector is in the sidebar AI panel.
4. Rate limit and restore time are in the sidebar AI panel.
5. AI output is plain terminal text, not a chat bubble.
6. User messages are right-aligned command cards.
7. Use black/white/gray as base colors.
8. Use green/orange/red only for success/warning/error states.
9. Keep the UI calm and minimal.
10. Avoid unnecessary labels, dashboards, and widgets.

15. Version 1 build checklist

- Create Next.js app and deploy to Vercel.
- Build the minimal CLI-terminal layout.
- Add sidebar with chat history and AI panel.
- Add clean composer with only chatbox and send icon.
- Implement localStorage sessions and settings.
- Add Instructions modal with save/reset and presets.
- Create Cloudflare Worker /api/analyze endpoint.
- Add NVIDIA NIM API call.
- Add OpenRouter fallback logic.
- Add clean error handling and rate-limit display in sidebar.
- Test with a public/signed video URL in the prompt.

16. Do not build yet

- User accounts

- Database sync
- Video upload
- Billing
- Clip export
- Team workspaces
- Complex analytics dashboards

17. Final product summary

Alaws lang. is a minimal CLI-style video analysis chat app.

The user pastes a video URL into a prompt.
The assistant analyzes the video and responds as terminal output.
The UI uses black and white, with green/orange/red only for status.
Chat history is stored locally for the free first version.
Model/provider controls live in the sidebar AI panel.
The composer stays clean: just a text box and send icon.
NVIDIA NIM is the primary provider.
OpenRouter free Gemma models are fallback providers.
Cloudflare Worker hides API keys and handles fallback logic.
Next.js on Vercel hosts the frontend.